



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование

WebSocke
f

WebSocket + Java



Проект



Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Java ☐ Kotlin ☐ Groovy
☒ Maven

Language

Spring Boot

☒ 3.4.0 (SNAPSHOT) ☐ 3.4.0 (RC1) ☐ 3.3.6 (SNAPSHOT) ☐ 3.3.5
☐ 3.2.12 (SNAPSHOT) ☐ 3.2.11

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ Jar ☐ War

Java ☐ 23 ☒ 21 ☐ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

WebSocket

MESSAGING

Build Servlet-based WebSocket applications with SockJS and STOMP.

Spring for RabbitMQ

MESSAGING

Gives your applications a common platform to send and receive messages, and your messages a safe place to live until received.



Цель реализации

- Мы рассмотрим реализацию уведомлений
 - Несмотря на то, что они текстовые, вы можете использовать как STOMP, так и базовый протокол WS по умолчанию
- Стоит отметить, что распространенный на Spring кейс – использование SocketJS – обертки над протоколом WS, часто используемый вместе со STOMP для реализации чатов, уведомлений и т.д.
 - Для чатов можно рассмотреть прямую реализацию через RabbitMQ



Конфигурация

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").setAllowedOriginPatterns("*").withSockJS();
    }
}
```



NotificationService

```
@Service
public class NotificationService {

    @Autowired
    private SimpMessagingTemplate template;

    public void sendNotification(String message) {
        template.convertAndSend("/topic/notifications", message);
    }
}
```



Main

```
@SpringBootApplication
public class DemowsApplication {
    static final String queueName = "firstQueue";

    @Autowired
    private NotificationService notificationService;

    @Bean
    public Queue myQueue() {
        return new Queue(queueName, false);
    }

    // ...
}
```



Main

```
@RabbitListener(queues = queueName)
    public void listen(String message) {
        System.out.println("Message read from firstQueue : " + message);
        notificationService.sendNotification(message);
    }

    public static void main(String[] args) {
        SpringApplication.run(DemowsApplication.class, args);
    }
}
```



Frontend

```
<!DOCTYPE html>
<html>
<head>
  <title>WebSocket Notifications</title>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/sockjs-client/1.6.1/sockjs.min.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/stomp.js/2.3.3/stomp.min.js"></script>
</head>
<body>
  <div id="notifications"></div>
  <script>
    var socket = new SockJS('http://localhost:8082/ws');
    var stompClient = Stomp.over(socket);

    . . .
```

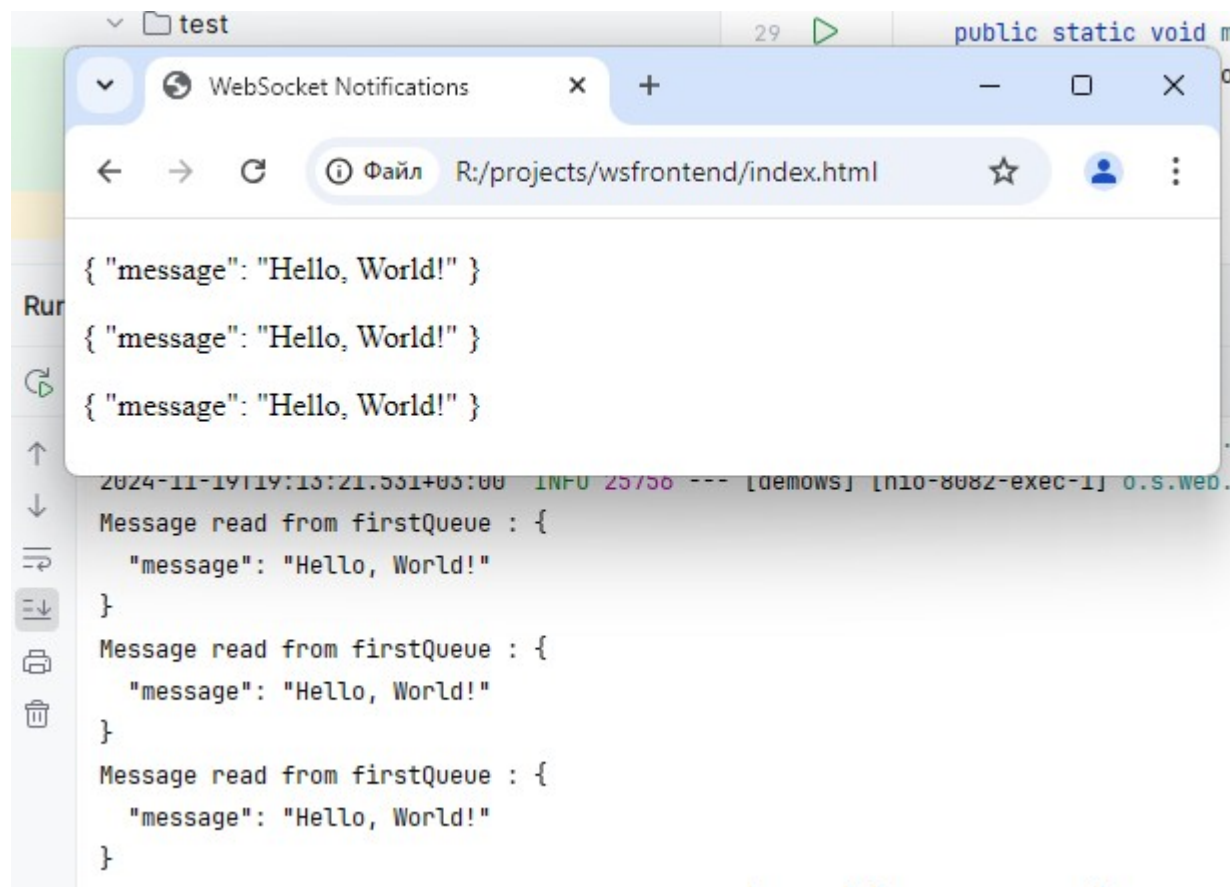


Frontend

```
stompClient.connect({}, function(frame) {  
    stompClient.subscribe('/topic/notifications', function(notification) {  
        var notifications = document.getElementById('notifications');  
        var message = document.createElement('p');  
        message.appendChild(document.createTextNode(notification.body));  
        notifications.appendChild(message);  
    });  
});  
</script>  
</body>  
</html>
```



Demo





Будут вопросы? Не стесняйтесь!

